Destination-based policy enforcement in Mixnets

Extending Sphinx Packets with Anonymous Credential

Aurélien Chassagne[1 [0009−0008−7207−868X]], Iness Ben Guirat[1 [0000−0002−8766−594X]], Jan Tobias Mühlberg[1 [0000−0001−5035−0576]], and Jean-Michel Dricot[1 [0000−0002−8539−9940]]

Université Libre de Bruxelles (ULB), Brussels, Belgium

Abstract Mixnets provide strong communication anonymity by hiding the relationship between senders and receivers. However, large-scale deployment introduces new operational challenges, as network operators may need to enforce policies on reachable destinations, for example to support service authorization, resource management, or operational agreements. Existing approaches perform destination verification at the exit gateway, where the final destination is available, but this occurs only after the traffic has already consumed the resources of the mixnet. This paper introduces an extension of the Sphinx packet format enabling privacy-preserving destination policy enforcement. Our approach integrates destination-bound credentials into the packet format, allowing a verifier to check the validity of an authority-issued credential and its binding to the encrypted final destination, without revealing the destination itself. This enables early rejection of unauthorized traffic while preserving destination privacy. Our work provides a mechanism to combine destination-based policy enforcement with the anonymity properties of mixnets, avoiding the need to reveal destinations for authorization purposes.

Keywords: Sphinx · Anonymous Credentials · Mixnet · Decentralized networks.

1 Introduction

Mixnets are a fundamental technology for privacy-preserving communication. By forwarding messages through a sequence of cryptographic relays, they make it difficult for adversaries to link senders and receivers while providing strong anonymity guarantees. The Sphinx packet format provides an efficient and secure foundation for such systems, offering compact packet processing, integrity protection, and unlinkability properties.

However, deploying mixnets at large scale raises challenges beyond cryptographic anonymity. A practical communication infrastructure may need to enforce policies related to service authorization, resource usage, or operational agreements. For example, a mixnet provider may offer subscription-based access

where users pay for the ability to anonymously communicate with a set of authorized destinations. Such a model requires a mechanism to verify that traffic is directed toward an authorized destination without revealing it.

Current mixnet architectures provide limited support for such requirements. Since intermediate mixnodes do not learn the final destination during packet processing, they cannot verify whether the destination satisfies the network's policies. A possible solution is to perform verification at the exit gateway, once the final destination information is available. However, this postpones policy enforcement until the traffic has already traversed the mixnet and consumed bandwidth and relay resources. The key challenge is therefore to enable early rejection of unauthorized traffic without requiring intermediate mixnodes to learn the destination they are forwarding toward.

This work explores a different approach: enabling privacy-preserving policy enforcement through destination-bound credentials. We propose an extension of the Sphinx packet format that enables a verifier to validate an authority-issued credential and verify its binding to the encrypted final destination, without revealing the destination itself.

Our goal is not to introduce destination surveillance into mixnets, but to enable privacy-preserving guarantees that communications are directed toward authorized endpoints. Such a mechanism provides a middle ground between unrestricted anonymous transport and destination disclosure for policy enforcement.

2 Proposed Scheme

This section presents the proposed protocol and its integration into the Sphinx packet format. Figure 1 provides an overview of the interactions between the client, the authorities, and the mixnet.

Our protocol consists of four steps: (1) the client requests destination-specific credential shares from a set of authorities, (2) the authorities verify the request according to the network policy and issue *partial credentials*, (3) the client aggregates the partial credentials into a *destination-bound credential*, and (4) when constructing a Sphinx packet, the client rerandomizes and binds the credential to the packet header, producing a *header-bound credential* that can be verified by mixnodes without revealing the destination.

DSphinx header. The proposed scheme is based on an adaptation of the Sphinx packet format, called DSphinx, first introduced in []. The key distinction is that each element in the header is independently encoded as an elliptic curve (EC) point. Consequently, the DSphinx header is structured as a sequence of fixed-size chunks, as illustrated in Fig. **??**, where each chunk corresponds to a single header element represented by an EC point. Each network node along the packet route contributes for two chunks: one encoding its address and another encoding a per-hop integrity tag. Therefore, a route of length m is represented by $n = 2m + 1$ chunks, accounting for two chunks per network node and one additional chunk corresponding to the final destination. Some routing information, such as

Cite DSphinx

If places allow, out a figure specific for the header

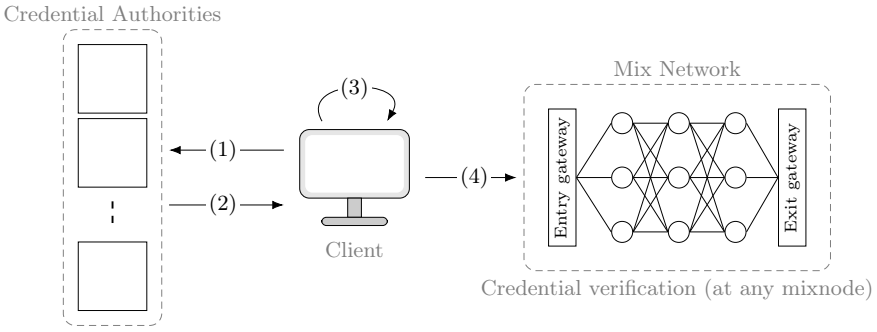

Figure 1: Overview of the credential issuance and header binding. The client obtains *partial credentials* from a set of authorities, aggregates them into a *destination-bound credential*, and finally rerandomizes and binds the credential to a Sphinx packet header, producing a *header-bound credential*.

IP addresses, must be reversibly encoded as elliptic curve points. To this end, DSphinx uses the Elligator mapping, which produces points computationally indistinguishable from randomly sampled group elements. .

To update no more Elligator since incompatible with pairing curves

2.1 Threshold Multi-Authority Setup

Update "distributed key generation" using "verifiable secret sharing" (VSS)

We consider a distributed authority composed of A independent authorities, where any subset of at least t authorities can jointly issue valid credentials. During system initialization, the authorities execute a standard distributed key generation (DKG) protocol based on verifiable secret sharing (VSS), resulting in a threshold signing key without ever reconstructing the corresponding secret.

More precisely, each authority A_k is associated with a public identifier $x_k \in \mathbb{Z}_q$ and receives a secret signing share y_k. Together, these shares define a secret polynomial f of degree $t-1$, whose constant coefficient $y = f(0)$ is the global signing key. While no individual authority learns y, any subset of at least t authorities can jointly sign any elliptic-curve point P as $\widetilde{P} = yP$ by combining their partial signature $(x_k, y_k P)$ using Lagrange interpolation at $x = 0$.

The authorities jointly authenticate the public parameters, consisting of the system public key $Y = yG'$, the public points $\widetilde{N}_i = yN_i$ of the M mixnodes in the network, and the chunk generators $\widetilde{G}_j = yG_j$.

$$\mathsf{pp} = \left\{ Y,\ \{\widetilde{N}_i\}_{i=1}^{M},\ \{\widetilde{G}_j\}_{j=1}^{n} \right\}. \tag{1}$$

2.2 Credential Issuance

Credential issuance is performed once for each destination and allows a client to obtain a *destination-bound* credential without revealing the destination to the authorities. The issuance process consists of two steps: partial credential issuance and its aggregation.

Partial Credential Issuance The client samples a random blinding factor $r \in \mathbb{Z}_q$ and computes the blinded representation of the destination Δ as $R = r\Delta$. The client then contacts at least t authorities and submits R together with a proof that the requested destination satisfies the applicable authorization policy.

The policy and the corresponding proof are application-dependent and are therefore outside the scope of this work. In particular, the proof may incorporate arbitrary authorization mechanisms supported by the deployment, such as a zero-knowledge proof of payment, without requiring the authorities to learn the destination itself. Our scheme only requires that the authorities can verify such a proof before issuing a credential.

Upon successful verification, each contacted authority A_k uses its secret signing share y_k to compute a *partial credential* $C_k = y_k R$, which is bound to the blinded destination R.

Credential Aggregation After collecting valid partial credentials from at least t authorities, the client combines them using Lagrange interpolation, obtaining an aggregated credential C_R bound to the blinded representation R. The client then removes the blinding factor and obtains the *destination-bound credential* $C = r^{-1}C_R = y\Delta$. Since the blinding factor r is known only by the client, the authorities learn neither Δ nor the resulting destination-bound credential during the issuance process.

2.3 Header Construction & Credential Binding

The header construction proceeds in four phases; a route and secrets derivation phase, a preprocessing phase that generates filler chunks, followed by a layer-by-layer encryption phase applied in reverse order, from the last hop to the first. Finally, the *destination-bound credential* is rerandomized and bound to the header, producing a *header-bound credential* that can be verified by any node without revealing the destination. The preprocessing and layer-by-layer encryption phases are respectively detailed in Alg. 1 and Alg. 2, and illustrated in Fig. 2 for $m = 3$. To ensure unlinkability, encryption is performed using a set of independently derived generators $\{G_j\}_{j=1}^{n}$, one per chunk in the header, such that no scalar relationship between any pair of generators is known.

Route and Secrets derivation. To send a message to a chosen destination Δ, for which he has the corresponding destination-bound credential $y\Delta$, the client selects a route $\{N_i\}_{i=1}^{m}$ and generates an initial nonce x_1. Using this nonce and the public keys of the nodes along the route, the client derives a sequence of shared secrets $\{s_i\}_{i=1}^{m}$ according to Eq. (2), yielding a hop-by-hop key derivation.

$$\{s_i\}_{i=1}^{m} \quad \begin{cases} \alpha_{\mathrm{i}} = x_i\, G, \\ s_{\mathrm{i}} = \mathsf{Decode}(x_i\, K_{\mathrm{i}}), \\ x_{i+1} = s_i\, x_i \pmod{p}, \end{cases} \tag{2}$$

Here, $K_\mathrm{i} = k_\mathrm{i} G$ is the public key of the i^{th} node in the path, p is the order of the prime subgroup, and $\mathsf{Decode}(\cdot)$ denotes the inverse mapping from elliptic curve points to scalars. The shared secret s_i acts as a per-hop pseudorandom update that derives the next blinding factor x_{i+1}, ensuring that the cryptographic elements α_i remain unlinkable across layers.

Preprocessing Phase. The preprocessing phase prepares a vector of filler chunks ϕ according to (3). These filler chunks ensure that after each encryption layer the last two chunks evaluate to the identity element and can thus be safely truncated, ensuring a constant header size and reversibility when processed by the network nodes. The detailed procedure is specified in Alg. 1, which takes as input the set of shared secrets $\{s_i\}$ and the set of chunk generators $\{G_j\}$, and outputs $n-1$ filler chunks, where $n = 2m+1$ is the header size for a path of length m.

$$\phi_j = -\sum_{i=1}^{m} s_i \, G_{\,n+j-2i}, \qquad 1 \le j \le n-1. \tag{3}$$

Layered Encryption Phase. The header is initialized by concatenating the destination chunk Δ with the filler chunks ϕ, producing a n-chunks header. Encryption is then applied iteratively in reverse path order, from $i = m$ down to $i = 1$, as illustrated in Fig. **??** and detailed in Alg. 2.

1. **Chunk encryption:** Each chunk j is encrypted according Eq. (4). The last two chunks evaluate to the identity element due to the preprocessing phase and may therefore be discarded while preserving subsequent processing.

$$H_{ij} = H_{(i+1,\,j)} + s_\mathrm{i} G_\mathrm{j} \tag{4}$$

2. **Integrity computation:** Remaining chunks are used for the integrity tag:

$$\gamma_\mathrm{i} = \mathrm{HMAC}(\beta_\mathrm{i}, s_i) \tag{5}$$

3. **Prepend Node:** The node address N_i and the integrity tag γ_i are prepended to H_i, producing again a n-chunks header.

Credential Binding. Before transmitting a packet, the client transforms the destination-bound credential into a *header-bound credential*. This transformation binds the credential to the current Sphinx header and pseudorandomizes its value, so that the destination-bound credential is not directly exposed in the packet.

Let H denote the header-dependent group element used to bind the credential to the current Sphinx header. It consists of two components: the public points of the mixnodes along the route, excluding the first hop, and a shared-secret-dependent contribution. For a route of length m, we define:

$$H = \underbrace{\sum_{i=2}^{m} N_i}_{\text{mixnode contribution}} + \underbrace{\sum_{j=1}^{m} s_j \left(\sum_{k=1}^{m-j+1} G_{2k-1} \right)}_{\text{shared-secret contribution}}. \tag{6}$$

Then the header-bound credential (σ) is computed by adding the destination-bound credential ($y\Delta$) to the header-dependent component H multiplied by the global secret key y. The authenticated header-dependent component yH is computed by the client using the authenticated public parameters $\{\widetilde{N}_i\}_{i=2}^{m}$ and $\{\widetilde{G}_{2*j-1}\}_{j=1}^{m}$, which are derived from the global secret key y.

$$\sigma = y(\Delta + H) = y\Delta + yH, \tag{7}$$

The resulting credential is both bound to the current Sphinx header and pseudorandomized by the header-dependent components. In particular, two packets carrying credentials for the same destination result in different header-bound credentials since their header-dependent values differ.

Algorithm 1 Preprocessing

Input:

$[s_i]_{i=1..m}$	shared secrets
$[G_j]_{j=1..n}$	chunk generators

Output:

ϕ	filler chunks

$m \leftarrow pathLength$
$n \leftarrow 2m + 1$
$\phi \leftarrow$ list of $(n-1)$ identity points
for $i = m$ **down to** 1 **do**
  for $j = 2(m - i + 1)$ **to** n **do**
    $l \leftarrow 2i + j - n$
    $\phi[l] \leftarrow \phi[l] - s_{\mathrm{i}} G_{\mathrm{j}}$
  end for
end for
return ϕ

Algorithm 2 Encryption

Input:

Δ	Destination
$[N_i]_{i=1..m}$	Route

Output:

H	header chunks

$H \leftarrow [\,\Delta \,\|\, \phi\,]$
for $i = m$ **down to** 1 **do**
  for $j = 1$ **to** n **do**
    $H[j] \leftarrow H[j] + s_{\mathrm{i}} G_{\mathrm{j}}$
  end for
  $H \leftarrow H[1 : n-2]$
  $\gamma_{\mathrm{i}} \leftarrow \mathrm{HMAC}(H, s_{\mathrm{i}})$
  $H \leftarrow [\,N_{\mathrm{i}} \,\|\, \gamma_{\mathrm{i}} \,\|\, H\,]$
end for
return H

2.4 Packet Processing & Credential Verification

Upon receiving a packet, mixnode i extracts the cryptographic element α_{i}, the integrity tag γ_{i}, and the encrypted routing information β_{i}. As in the original Sphinx design, the node checks whether α_{i} has been previously observed since its last key rotation and discards duplicates to prevent replay attacks. The header is processed as illustrated in Fig. 3.

Credential Verification. First the mixnode verifies the header-bound credential σ using the system public key $Y = yG'$. The verification considers all header chunks that may contain the encrypted final destination (i.e. all odd-indexed β

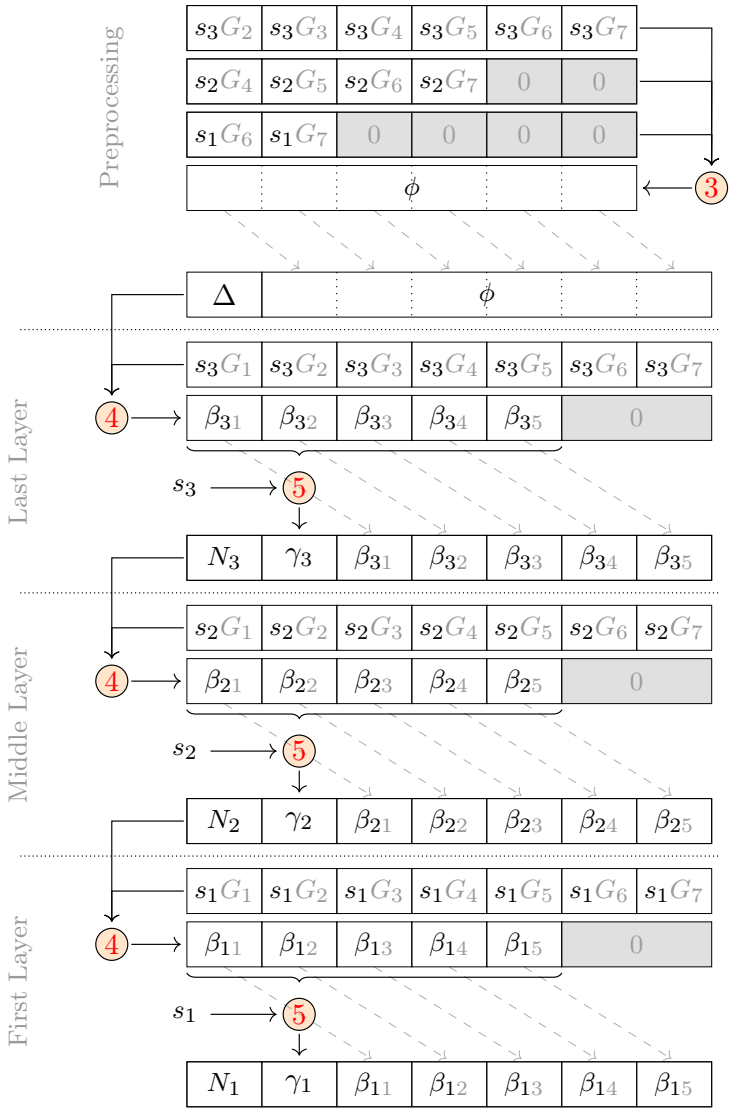

Figure 2: DSphinx header construction for a path of length $m = 3$ (without credential binding). Circled numbers (x) refers to Equation x in the paper.

chunks), and therefore does not require the mixnode to determine which chunk actually contains the destination. The credential is accepted if the following pairing equation holds; otherwise, the packet is discarded:

$$e(\sigma, G') = e\left(\sum_{i=1}^{m} \beta_{2\text{i-}1}, Y\right), \tag{8}$$

Since all candidate destination chunks are included in the verification, a successful verification establishes that σ is a valid authority-issued credential bound to the encrypted destination carried by the header, without revealing which candidate chunk contains that destination.

Integrity Check. The mixnode derives the shared secret s_i by multiplying the cryptographic element α_i with its private key k_i, and then hashing the resulting point into a scalar as in (9). The shared secret s_i is then used to verify the integrity tag in (10).

$$s_\text{i} = \text{HashToScalar}(k_\text{i}\,\alpha_\text{i}) \tag{9}$$

$$\gamma_\text{i} \stackrel{?}{=} \text{HMAC}(\beta_\text{i}, s_\text{i}) \tag{10}$$

$$\tag{11}$$

Decryption. If verification succeeds, the node appends two identity chunks and decrypts the header as in (12). The first decrypted chunk $\beta'_{\text{i}1}$ reveals the next hop $N_{\text{i}+1}$. The second chunk $\beta'_{\text{i}2}$ contains the integrity tag $\gamma_{\text{i}+1}$ for the next hop, while the remaining chunks form the next encrypted routing information $\beta_{\text{i}+1}$.

$$\beta'_{\text{ij}} = \beta_{\text{ij}} - s_\text{i} G_\text{j} \tag{12}$$

Update. Finally, the node updates the cryptographic element according to (13). It also updates the credential according to (14) using the authenticated next hop public point $\widetilde{N}_{i+1}$. Now the packet has been processed by mixnode i, it can be shuffled, delayed and forwarded to the next hop.

$$\alpha_{\text{i}+1} = s_i\,\alpha_\text{i} \tag{13}$$

$$\sigma_{\text{i}+1} = \sigma_\text{i} - s_i \left(\sum_{k=1}^{m+1} \widetilde{G}_{2k-1}\right) - \widetilde{N}_{i+1} \tag{14}$$

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

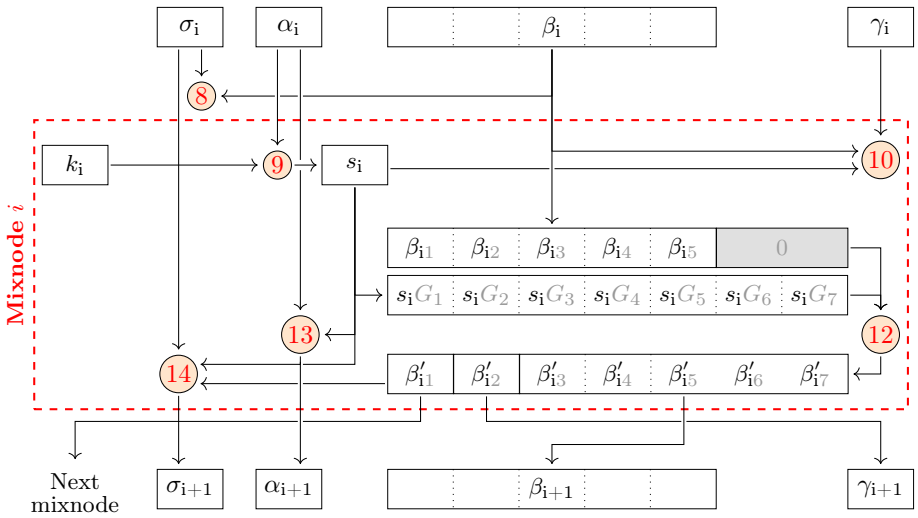

Figure 3: DSphinx header of path length $m = 3$ and its credential processed by mixnode i. Circled numbers (x) refers to Equation x in the paper.